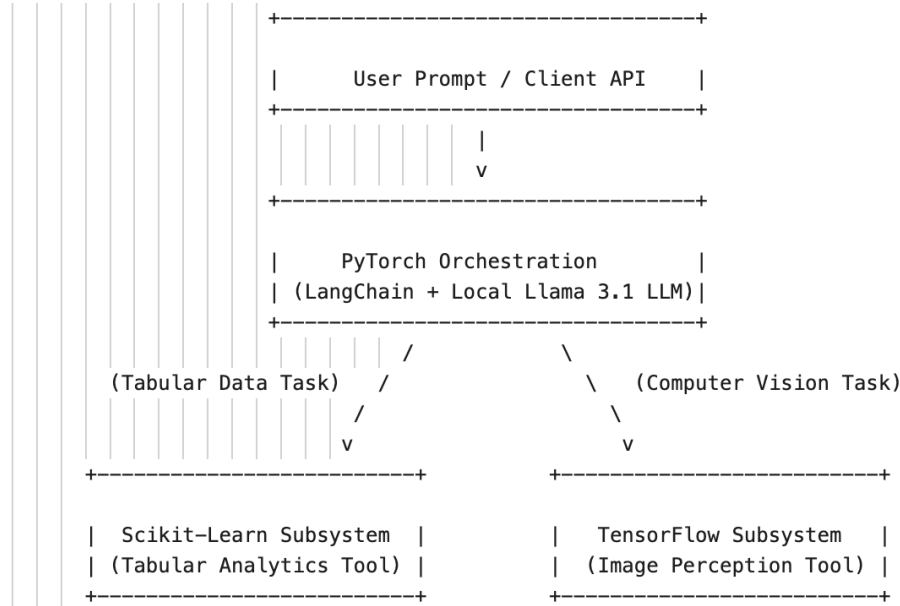


OmniMind Multi-Framework AI Agent Ecosystem

OmniMind: Multi-Framework AI Agent Ecosystem

An enterprise-grade, locally deployed AI Agent architecture that unifies three distinct machine learning paradigms. The system leverages an asynchronous orchestration engine to route tasks to framework-specific specialized subsystems based on the type of incoming data.

System Architecture



Architectural Breakdown

1. **Orchestration Layer (PyTorch):** Utilizes a local LLM via Hugging Face/Ollama. This layer processes natural language intents, manages state, and dynamically selects tools using function-calling mechanisms.
2. **Analytics Layer (Scikit-learn):** A highly efficient, CPU-bound pipeline optimized for memory-mapped structured datasets. It provides deterministic, rapid inference for classification and regression tasks without deep learning overhead.
3. **Perception Layer (TensorFlow):** A parallelized, GPU-optimized deep convolutional pipeline designed for unstructured computer vision payloads.

OmniMind Project

OmniMind is a local, multi-framework AI agent system that routes user requests to specialized tools:

- Tabular analytics with Scikit-learn
- Vision inference with TensorFlow (MobileNetV2 workflow)
- Agent orchestration with LangChain + Ollama

Technology Stack

- Language: Python 3
- Agent Runtime: LangChain, LangChain Ollama
- Local LLM Provider: Ollama (configured model: llama3.1)
- Tabular ML: Scikit-learn, Joblib, NumPy
- Vision ML: TensorFlow / Keras (MobileNetV2 transfer learning)
- Image IO: Pillow
- Environment: venv

Project Structure (With Comments)

```
```text
omnimind-project/
├── config.py # Central paths and runtime model configuration
├── requirements.txt # Python dependencies
├── train_artifacts.py # Main artifact/training pipeline (sklearn + vision fine-tune)
├── train_artifacts_mock.py # Alternate/mock training script variant
├── README.md # Project documentation
|
├── data/
| ├── sample_image.jpeg # Default sample image path used by orchestrator
| ├── sample_image_dog.jpg # Sample dog image (dataset/bootstrap support)
| ├── sample_image_dots.jpg # Sample dots image (dataset/bootstrap support)
| └── models/
| ├── housing_model.joblib # Serialized Scikit-learn tabular model artifact
| ├── vision_model.keras # Serialized TensorFlow vision model artifact
| └── vision_labels.json # Label metadata for custom fine-tuned classes
| └── vision/
| ├── README.md # Vision dataset format and naming rules
| ├── train/ # Labeled training images by class folder
| └── val/ # Labeled validation images by class folder
└── src/
 ├── __init__.py # Package exports for src
```

```

└─ orchestrator.py # LangChain/Ollama agent that routes to tools
└─ tools/
 └─ __init__.py # Tool package exports
 └─ analytics.py # Real-estate valuation tool (tabular regression)
 └─ perception.py # Vision inference tool (top-k predictions)
 └─ perception_mock.py # Alternate/mock vision tool implementation
...

```

#### 1. Configuration Module (config.py)

This file ensures absolute paths and environmental settings are unified across all frameworks.

#### 2. Mock Training Script (train\_artifacts.py)

This script generates a valid scikit-learn linear regressor and a TensorFlow Convolutional Neural Network (CNN), serializing them to disk so the agent has functional tools immediately.

#### 3. Analytics Tool Layer (src/tools/analytics.py)

This module manages the Scikit-learn sub-engine. It includes structural error handling to verify the presence of model binaries before running standard CPU metrics.

#### 4. Perception Tool Layer (src/tools/perception.py)

This module manages the TensorFlow sub-engine, wrapping native array preprocessing operations inside safe execution bounds.

#### 5. Orchestration Layer (src/orchestrator.py)

This module uses a PyTorch-backed language model framework (langchain interfacing with local Ollama models) to bind the custom tools and route work dynamically based on incoming intent patterns.

### ## File-by-File Responsibilities

#### ### Root Files

- config.py
  - Defines canonical paths for models, datasets, and sample images
  - Defines LOCAL\_LLM\_MODEL used by the orchestrator
- requirements.txt
  - Lists all required Python libraries for orchestration, ML, and image handling
- train\_artifacts.py
  - Trains and saves the Scikit-learn housing model
  - Builds/fine-tunes TensorFlow MobileNetV2 when a valid dataset exists
  - Validates dataset quality (class alignment, minimum distinct images, no train/val leakage)
  - Saves vision labels metadata and prints per-class validation accuracy
- train\_artifacts\_mock.py
  - Mock/alternate training path used for experimentation or legacy behavior
- README.md
  - Main project documentation (this file)

### ### Source Package

- src/\_\_\_init\_\_\_py
  - Exposes package-level imports for the src module
- src/orchestrator.py
  - Initializes ChatOllama model and LangChain agent
  - Registers analytics and vision tools
  - Routes user prompts to the correct subsystem
- src/tools/\_\_\_init\_\_\_py
  - Re-exports tool entry points for clean imports
- src/tools/analytics.py
  - Loads tabular model artifact
  - Parses tool input in square\_foot,bedrooms format
  - Returns formatted valuation output
- src/tools/perception.py
  - Loads TensorFlow model artifact and optional custom labels
  - Preprocesses input image and performs inference
  - Returns top prediction and top-3 confidence output
- src/tools/perception\_mock.py
  - Mock/alternate vision behavior for testing

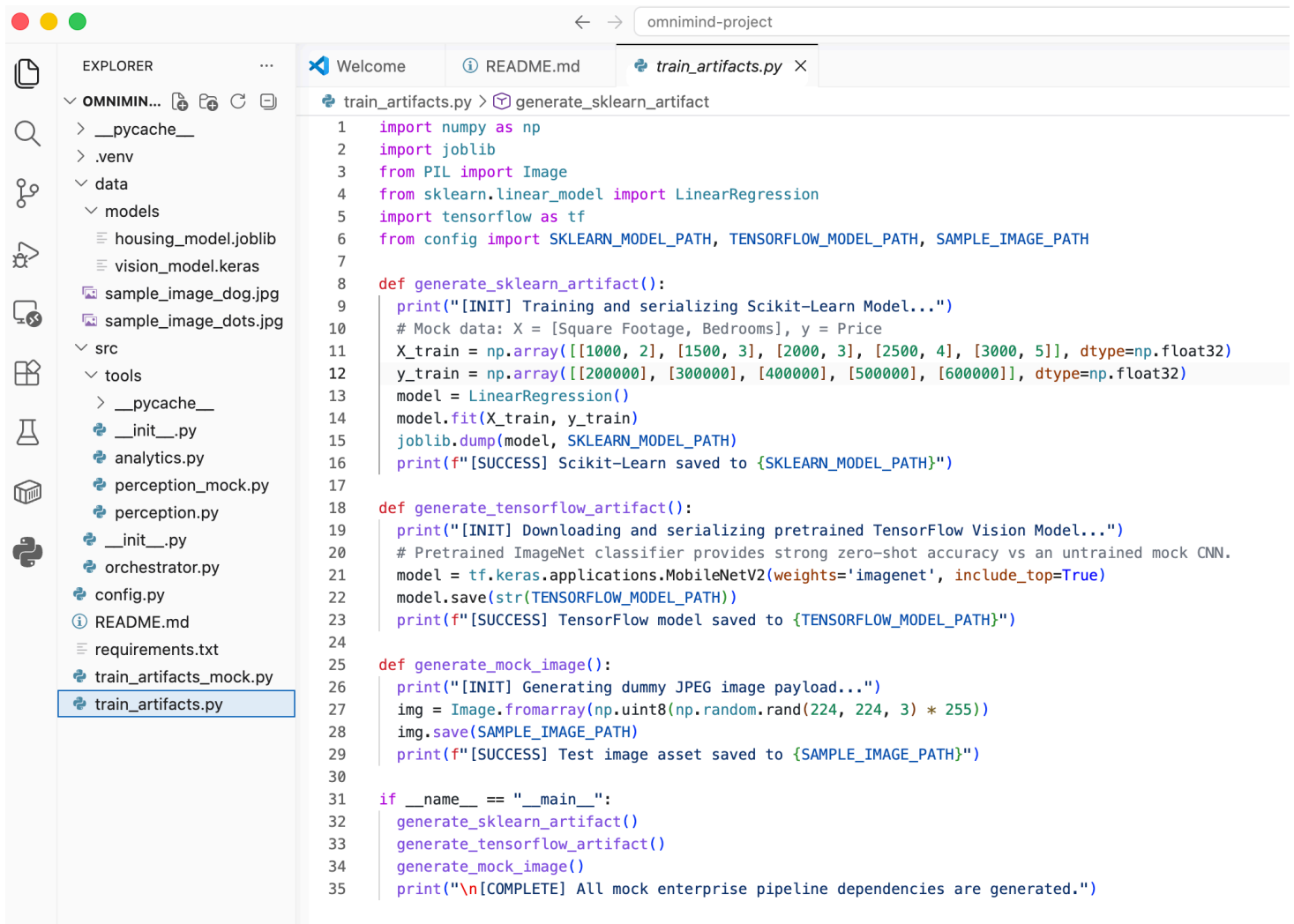
### ### Data and Artifacts

- data/models/housing\_model.joblib
  - Trained tabular model used by analytics tool
- data/models/vision\_model.keras
  - Trained or fallback vision model used by perception tool
- data/models/vision\_labels.json
  - Indicates custom vs imagenet label source and class names
- data/vision/train and data/vision/val
  - Class-folder datasets used for fine-tuning and validation
- data/vision/README.md
  - Dataset layout, naming conventions, and usage notes

### ## How the System Works (Summary)

1. You generate artifacts with train\_artifacts.py.
2. The orchestrator starts and loads the local Ollama LLM.
3. User prompt enters the orchestrator.
4. Agent selects analytics or perception tool.
5. Tool loads its model artifact and returns a prediction.

Here is the training artifacts code that seeds the [analytics tool \(Scikit-Learn Model\)](#), seeds the perception tool (TensorFlow Vision Model), and creates a default test image (dot pixels):



```
1 import numpy as np
2 import joblib
3 from PIL import Image
4 from sklearn.linear_model import LinearRegression
5 import tensorflow as tf
6 from config import SKLEARN_MODEL_PATH, TENSORFLOW_MODEL_PATH, SAMPLE_IMAGE_PATH
7
8 def generate_sklearn_artifact():
9 print("[INIT] Training and serializing Scikit-Learn Model...")
10 # Mock data: X = [Square Footage, Bedrooms], y = Price
11 X_train = np.array([[1000, 2], [1500, 3], [2000, 3], [2500, 4], [3000, 5]], dtype=np.float32)
12 y_train = np.array([[200000], [300000], [400000], [500000], [600000]], dtype=np.float32)
13 model = LinearRegression()
14 model.fit(X_train, y_train)
15 joblib.dump(model, SKLEARN_MODEL_PATH)
16 print(f"[SUCCESS] Scikit-Learn saved to {SKLEARN_MODEL_PATH}")
17
18 def generate_tensorflow_artifact():
19 print("[INIT] Downloading and serializing pretrained TensorFlow Vision Model...")
20 # Pretrained ImageNet classifier provides strong zero-shot accuracy vs an untrained mock CNN.
21 model = tf.keras.applications.MobileNetV2(weights='imagenet', include_top=True)
22 model.save(str(TENSORFLOW_MODEL_PATH))
23 print(f"[SUCCESS] TensorFlow model saved to {TENSORFLOW_MODEL_PATH}")
24
25 def generate_mock_image():
26 print("[INIT] Generating dummy JPEG image payload...")
27 img = Image.fromarray(np.uint8(np.random.rand(224, 224, 3) * 255))
28 img.save(SAMPLE_IMAGE_PATH)
29 print(f"[SUCCESS] Test image asset saved to {SAMPLE_IMAGE_PATH}")
30
31 if __name__ == "__main__":
32 generate_sklearn_artifact()
33 generate_tensorflow_artifact()
34 generate_mock_image()
35 print("\n[COMPLETE] All mock enterprise pipeline dependencies are generated.")
```

## Generate the serialized models required for the tools layer

The screenshot shows a VS Code editor window with the following components:

- EXPLORER:** A file tree on the left showing the project structure. The `src` directory is expanded, showing files like `__init__.py`, `analytics.py`, `perception.py`, `orchestrator.py`, and `config.py`. The `tools` directory is also visible, containing `__init__.py`, `analytics.py`, `perception.py`, `orchestrator.py`, and `config.py`.
- README.md:** The main editor shows the content of the `README.md` file. It includes a list of dependencies (requirements.txt) and instructions for generating serialized models. The instructions are as follows:
  - 1. Install system dependencies: `bash: python -m pip install -r requirements.txt`
  - 2. Generate the serialized models required for the tools layer: `bash: python train_artifacts.py`
  - 3. Run the orchestration engine: `bash: python src/orchestrator.py`
- TERMINAL:** The terminal window at the bottom shows the output of the commands. It displays the following messages:
  - [COMPLETE] All mock enterprise pipeline dependencies are generated.
  - [INIT] Training and serializing Scikit-Learn Model...
  - [SUCCESS] Scikit-Learn saved to /Users/christisanderson/omnimind-project/data/models/housing\_model.joblib
  - [INIT] Building and serializing TensorFlow Vision Model...
  - [SUCCESS] TensorFlow model saved to /Users/christisanderson/omnimind-project/data/models/vision\_model.keras
  - [INIT] Generating dummy JPEG image payload...
  - [SUCCESS] Test image asset saved to /Users/christisanderson/omnimind-project/data/sample\_image.jpg
  - [COMPLETE] All mock enterprise pipeline dependencies are generated.

## Start the Orchestrator

### Example of SYSTEM\_CRITICAL\_FAILURE (need to start Ollama server—model not available)The

```
(.venv) %n@%m %1~ %#
(.venv) %n@%m %1~ %# python src/orchestrator.py
[ORCHESTRATOR] Initializing LLM runtime [llama3] via Ollama...

[USER PROMPT]: I have an asset transaction profile. Can you predict the valuation score of a home listing featuring 2400 square feet containing 4 bedrooms?

[values] {'messages': [HumanMessage(content='I have an asset transaction profile. Can you predict the valuation score of a home listing featuring 2400 square feet containing 4 bedrooms?', additional_kwargs={}, response_metadata={}, id='17cd889f-088f-410e-993b-0d2997a7e63e')]}

[SYSTEM CRITICAL FAILURE]: Architecture initialization halted. Reason: registry.ollama.ai/library/llama3:latest does not support tools (status code: 400)
[TIP] Ensure your local Ollama server is running and the model is available.
(.venv) %n@%m %1~ %#
```

## Orchestrator agent is using tool-calling, but llama3 does not support tools. Using GitHub Copilot to help resolve the error (switch to llama3.1).

omnimind-project

EXPLORER

- OMNIMIND-PROJECT
  - \_\_pycache\_\_
  - .venv
  - data
    - models
    - housing\_model.joblib
    - vision\_model.keras
    - sample\_image.jpg
  - src
  - tools
    - \_\_pycache\_\_
    - \_\_init\_\_.py
    - analytics.py
    - perception.py
    - orchestrator.py
    - config.py
  - README.md
  - requirements.txt
  - train\_artifacts.py

README.md

```
Imaging
ollama pull llama3
ollama serve
4. Run the orchestration engine: bash:
python src/orchestrator.py

Part 2: Implementation Code
Create configuration and training routines to generate our mock AI weights, followed by the specific core layers.

1. Configuration Module (config.py)
This file ensures absolute paths and environmental settings are unified across all frameworks.
2. Mock Training Script (train_artifacts.py)
This script generates a valid scikit-learn linear regressor and a TensorFlow Convolutional Neural Network (CNN), serializing them to disk so the agent has functional tools immediately.
3. Analytics Tool Layer (src/tools/analytics.py)
This module manages the Scikit-learn sub-engine. It includes structural error handling to verify the presence of model binaries before running standard CPU metrics.
4. Perception Tool Layer (src/tools/perception.py)
This module manages the TensorFlow sub-engine, wrapping native array preprocessing operations inside safe execution bounds.
5. Orchestration Layer (src/orchestrator.py)
This module uses a PyTorch-backed language model framework (langchain interfacing with local Ollama
```

PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL

```
(.venv) christisanderson@Mac omnimind-project % bash
(.venv) %n@%m %1~ %#
(.venv) %n@%m %1~ %# python src/orchestrator.py
[ORCHESTRATOR] Initializing LLM runtime [llama3] via Ollama...

[USER PROMPT]: I have an asset transaction profile. Can you predict the valuation score of a home listing featuring 2400 square feet containing 4 bedrooms?

[values] {'messages': [HumanMessage(content='I have an asset transaction profile. Can you predict the valuation score of a home listing featuring 2400 square feet containing 4 bedrooms?', additional_kwargs={}, response_metadata={}, id='17cd889f-088f-410e-993b-0d2997a7e63e')]}

[SYSTEM CRITICAL FAILURE]: Architecture initialization halted. Reason: registry.ollama.ai/library/llama3:latest does not support tools (status code: 400)
[TIP] Ensure your local Ollama server is running and the model is available.
(.venv) %n@%m %1~ %# python src/orchestrator.py
[ORCHESTRATOR] Initializing LLM runtime [llama3] via Ollama...

[USER PROMPT]: I have an asset transaction profile. Can you predict the valuation score of a home listing featuring 2400 square feet containing 4 bedrooms?

[values] {'messages': [HumanMessage(content='I have an asset transaction profile. Can you predict the valuation score of a home listing featuring 2400 square feet containing 4 bedrooms?', additional_kwargs={}, response_metadata={}, id='d9b56dba-2431-47ef-b0c8-afc73f5e8236')]}

[SYSTEM CRITICAL FAILURE]: Architecture initialization halted. Reason: registry.ollama.ai/library/llama3:latest does not support tools (status code: 400)
[TIP] Ensure your local Ollama server is running and the model is available.
(.venv) %n@%m %1~ %#
```

CHAT

PROJECT FRONT END NECESSITY  
[TIP] Ensure your local Ollama server is running and the model is available.  
(.venv) %n@%m %1~ %#

I traced the failure to model capability mismatch, and I'll explain exactly why it happens plus the fastest way to fix it.

You are getting that error because your agent is now using tool-calling, but the Ollama model you selected does not support tools.

What is happening:

- orchestrator.py builds an agent with tools.
- LangChain sends a tool-enabled request to Ollama.
- Ollama routes it to llama3:latest.
- That model returns 400 with "does not support tools."

So this is not a "server is down" issue. The server is up, model is present, but model capability is wrong for this agent setup.

Fast fix:

- Pull a tool-capable model in Ollama (for example, llama3.1).
- Change LOCAL\_LLM\_MODEL in config.py from llama3 to a tool-capable model name you pulled.
- Re-run:  
python train\_artifacts.py  
python orchestrator.py

If you want, I can update config.py now and also improve the error tip in orchestrator.py so it explicitly says "model does not support tools" instead of only suggesting server availability.

+ README.md  
Describe what to build  
+ Auto

Local Default Approvals

Ln 71, Col 29 Spaces: 4 UTF-8 LF {} Markdown



Now that llama3.1 is running successfully start the testing.

For this prompt, [Orchestrator routed to Tabular Analytics Tool](#) and gave proper answer based on seeded train data:

PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL

```
(.venv) %n@m %l~ %# python src/orchestrator.py
[ORCHESTRATOR] Initializing LLM runtime [llama3.1] via Ollama...
```

```
[USER PROMPT]: I have an asset transaction profile. Can you predict the valuation score of a home listing featuring 2400 square feet containing 4 bedrooms?
```

```
[values] {'messages': [HumanMessage(content='I have an asset transaction profile. Can you predict the valuation score of a home listing featuring 2400 square feet containing 4 bedrooms?', additional_kwargs={}, response_metadata={}, id='f1f52d53-6721-4534-ba8c-d19178afa07f')]}
[updates] {'model': {'messages': [AIMessage(content='', additional_kwargs={}, response_metadata={'model': 'llama3.1', 'created_at': '2026-07-01T03:51:57.384821Z', 'done': True, 'done_reason': 'stop', 'total_duration': 3621415584, 'load_duration': 911896709, 'prompt_eval_count': 304, 'prompt_eval_duration': 1559532000, 'eval_count': 24, 'eval_duration': 1147377000, 'logprobs': None, 'model_name': 'llama3.1', 'model_provider': 'ollama'}, id='lc_run--019f1bce-02a2-7f82-ba43-c43fffc1cdd2-0', tool_calls=[{'name': 'run_real_estate_prediction', 'args': {'tool_input': '2400,4'}, 'id': '2d34aca6-0e19-4a88-b870-7e09a78909c7', 'type': 'tool_call'}], invalid_tool_calls=[], usage_metadata={'input_tokens': 304, 'output_tokens': 24, 'total_tokens': 328})]}}
```

```
[values] {'messages': [HumanMessage(content='I have an asset transaction profile. Can you predict the valuation score of a home listing featuring 2400 square feet containing 4 bedrooms?', additional_kwargs={}, response_metadata={}, id='f1f52d53-6721-4534-ba8c-d19178afa07f'), AIMessage(content='', additional_kwargs={}, response_metadata={'model': 'llama3.1', 'created_at': '2026-07-01T03:51:57.384821Z', 'done': True, 'done_reason': 'stop', 'total_duration': 3621415584, 'load_duration': 911896709, 'prompt_eval_count': 304, 'prompt_eval_duration': 1559532000, 'eval_count': 24, 'eval_duration': 1147377000, 'logprobs': None, 'model_name': 'llama3.1', 'model_provider': 'ollama'}, id='lc_run--019f1bce-1179-72f2-936c-b465c10ed151-0', tool_calls=[], invalid_tool_calls=[], usage_metadata={'input_tokens': 161, 'output_tokens': 27, 'total_tokens': 188})]}}
```

PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL

```
), ToolMessage(content='Scikit-Learn Subsystem: Estimated Valuation is $480,000.00', name='run_real_estate_prediction', id='f464cdcb-3cfe-4bb3-9563-f65ce362abfe', tool_call_id='2d34aca6-0e19-4a88-b870-7e09a78909c7')]}
[updates] {'model': {'messages': [AIMessage(content='Based on the tool's output, I predict that the valuation score of the home listing is approximately $480,000.00.', additional_kwargs={}, response_metadata={'model': 'llama3.1', 'created_at': '2026-07-01T03:51:59.390347Z', 'done': True, 'done_reason': 'stop', 'total_duration': 1828665583, 'load_duration': 101362542, 'prompt_eval_count': 161, 'prompt_eval_duration': 481265000, 'eval_count': 27, 'eval_duration': 1241544000, 'logprobs': None, 'model_name': 'llama3.1', 'model_provider': 'ollama'}, id='lc_run--019f1bce-1179-72f2-936c-b465c10ed151-0', tool_calls=[], invalid_tool_calls=[], usage_metadata={'input_tokens': 161, 'output_tokens': 27, 'total_tokens': 188})]}}
```

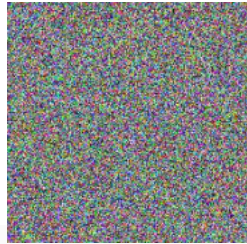
```
[values] {'messages': [HumanMessage(content='I have an asset transaction profile. Can you predict the valuation score of a home listing featuring 2400 square feet containing 4 bedrooms?', additional_kwargs={}, response_metadata={}, id='f1f52d53-6721-4534-ba8c-d19178afa07f'), AIMessage(content='', additional_kwargs={}, response_metadata={'model': 'llama3.1', 'created_at': '2026-07-01T03:51:57.384821Z', 'done': True, 'done_reason': 'stop', 'total_duration': 3621415584, 'load_duration': 911896709, 'prompt_eval_count': 304, 'prompt_eval_duration': 1559532000, 'eval_count': 24, 'eval_duration': 1147377000, 'logprobs': None, 'model_name': 'llama3.1', 'model_provider': 'ollama'}, id='lc_run--019f1bce-02a2-7f82-ba43-c43fffc1cdd2-0', tool_calls=[{'name': 'run_real_estate_prediction', 'args': {'tool_input': '2400,4'}, 'id': '2d34aca6-0e19-4a88-b870-7e09a78909c7', 'type': 'tool_call'}], invalid_tool_calls=[], usage_metadata={'input_tokens': 304, 'output_tokens': 24, 'total_tokens': 328}), ToolMessage(content='Scikit-Learn Subsystem: Estimated Valuation is $480,000.00', name='run_real_estate_prediction', id='f464cdcb-3cfe-4bb3-9563-f65ce362abfe', tool_call_id='2d34aca6-0e19-4a88-b870-7e09a78909c7'), AIMessage(content='Based on the tool's output, I predict that the valuation score of the home listing is approximately $480,000.00.', additional_kwargs={}, response_metadata={'model': 'llama3.1', 'created_at': '2026-07-01T03:51:59.390347Z', 'done': True, 'done_reason': 'stop', 'total_duration': 1828665583, 'load_duration': 101362542, 'prompt_eval_count': 161, 'prompt_eval_duration': 481265000, 'eval_count': 27, 'eval_duration': 1241544000, 'logprobs': None, 'model_name': 'llama3.1', 'model_provider': 'ollama'}, id='lc_run--019f1bce-1179-72f2-936c-b465c10ed151-0', tool_calls=[], invalid_tool_calls=[], usage_metadata={'input_tokens': 161, 'output_tokens': 27, 'total_tokens': 188})]}}
```

[AGENT FINAL RESPONSE]:

Based on the tool's output, I predict that the valuation score of the home listing is approximately \$480,000.00.



For this prompt, **Orchestrator** routed to **Image Perception Tool** and gave proper answer based on seeded train data (**dot image**):



```
[USER PROMPT]: We have ingested an unstructured binary media payload file. Please audit what is inside the asset located at file path: /Users/christisanderson/omnimind-project/data/sample_image.jpg
```

```
[values] {'messages': [HumanMessage(content='We have ingested an unstructured binary media payload file. Please audit what is inside the asset located at file path: /Users/christisanderson/omnimind-
```

PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL

```
'model_provider': 'ollama'}, id='lc_run--019f1bd6-9903-7d21-959c-325f056c8be5-0', tool_calls=[{'name': 'run_vision_inference', 'args': {'image_path': '/Users/christisanderson/omnimind-project/data/sample_image.jpg'}}, {'id': 'cb182516-b4eb-4891-a9bd-95dbfc7f706a', 'type': 'tool_call'}], invalid_tool_calls=[], usage_metadata={'input_tokens': 316, 'output_tokens': 35, 'total_tokens': 351}), ToolMessage(content='TensorFlow Inference Result: Top prediction -> 'velvet' with 42.92% confidence. Top-3: velvet (42.92%), wool (15.53%), bubble (10.02%).', name='run_vision_inference', id='d3cadfbc-0010-41b3-be04-00203adb33e3', tool_call_id='cb182516-b4eb-4891-a9bd-95dbfc7f706a')]]
```

```
[updates] {'model': {'messages': [AIMessage(content='Based on the vision tool's output, it appears that the image at the specified file path is a picture of velvet fabric. The top prediction from the model is "velvet" with a confidence level of 42.92%. The top three predictions are:\n\n1. Velvet (42.92%)\n2. Wool (15.53%)\n3. Bubble (10.02%)\n\nThis suggests that the image contains characteristics commonly associated with velvet, such as its texture and appearance.', additional_kwargs={}, response_metadata={'model': 'llama3.1', 'created_at': '2026-07-01T04:01:24.948403Z', 'done': True, 'done_reason': 'stop', 'total_duration': 5390821166, 'load_duration': 114129458, 'prompt_eval_count': 210, 'prompt_eval_duration': 794511000, 'eval_count': 96, 'eval_duration': 4475707000, 'logprobs': None, 'model_name': 'llama3.1', 'model_provider': 'ollama'}, id='lc_run--019f1bd6-a4c4-7f82-98f2-ec0536364ab7-0', tool_calls=[], invalid_tool_calls=[], usage_metadata={'input_tokens': 210, 'output_tokens': 96, 'total_tokens': 306})]}}
```

```
[values] {'messages': [HumanMessage(content='We have ingested an unstructured binary media payload file. Please audit what is inside the asset located at file path: /Users/christisanderson/omnimind-project/data/sample_image.jpg', additional_kwargs={}, response_metadata={}, id='ed7f30f6-005d-44b7-b56e-618a47a288da'), AIMessage(content='', additional_kwargs={}, response_metadata={'model': 'llama3.1', 'created_at': '2026-07-01T04:01:18.583357Z', 'done': True, 'done_reason': 'stop', 'total_duration': 2035095292, 'load_duration': 99587500, 'prompt_eval_count': 316, 'prompt_eval_duration': 322600000, 'eval_count': 35, 'eval_duration': 1606920000, 'logprobs': None, 'model_name': 'llama3.1', 'model_provider': 'ollama'}, id='lc_run--019f1bd6-9903-7d21-959c-325f056c8be5-0', tool_calls=[{'name': 'run_vision_inference', 'args': {'image_path': '/Users/christisanderson/omnimind-project/data/sample_image.jpg'}}, {'id': 'cb182516-b4eb-4891-a9bd-95dbfc7f706a', 'type': 'tool_call'}], invalid_tool_calls=[], usage_metadata={'input_tokens': 316, 'output_tokens': 35, 'total_tokens': 351}), ToolMessage(content='TensorFlow Inference Result: Top prediction -> 'velvet' with 42.92% confidence. Top-3: velvet (42.92%), wool (15.53%), bubble (10.02%).', name='run_vision_inference', id='d3cadfbc-0010-41b3-be04-00203adb33e3', tool_call_id='cb182516-b4eb-4891-a9bd-95dbfc7f706a'), AIMessage(content='Based on the vision tool's output, it appears that the image at the specified file path is a picture of velvet fabric. The top prediction from the model is "velvet" with a confidence level of 42.92%. The top three predictions are:\n\n1. Velvet (42.92%)\n2. Wool (15.53%)\n3. Bubble (10.02%)\n\nThis suggests that the image contains characteristics commonly associated with velvet, such as its texture and appearance.', additional_kwargs={}, response_metadata={'model': 'llama3.1', 'created_at': '2026-07-01T04:01:24.948403Z', 'done': True, 'done_reason': 'stop', 'total_duration': 5390821166, 'load_duration': 114129458, 'prompt_eval_count': 210, 'prompt_eval_duration': 794511000, 'eval_count': 96, 'eval_duration': 4475707000, 'logprobs': None, 'model_name': 'llama3.1', 'model_provider': 'ollama'}, id='lc_run--019f1bd6-a4c4-7f82-98f2-ec0536364ab7-0', tool_calls=[], invalid_tool_calls=[], usage_metadata={'input_tokens': 210, 'output_tokens': 96, 'total_tokens': 306})]}}
```

```
[AGENT FINAL RESPONSE]:
```

```
Based on the vision tool's output, it appears that the image at the specified file path is a picture of velvet fabric. The top prediction from the model is "velvet" with a confidence level of 42.92%. The top three predictions are:
```

1. Velvet (42.92%)
2. Wool (15.53%)
3. Bubble (10.02%)

```
This suggests that the image contains characteristics commonly associated with velvet, such as its texture and appearance.
```

```
(.venv) %n@m %1~ %# □
```

bash  
ollama

For this prompt, **Orchestrator routed to Image Perception Tool** and gave proper answer based on seeded train data (**dog image**):



```
..., additional_kwargs={}, response_metadata={'model': 'llama3.1', 'created_at': '2020-07-01T04:33:14.991927Z', 'done': True, 'done_reason': 'stop', 'total_duration': 6777721750, 'load_duration': 97461791, 'prompt_eval_count': 216, 'prompt_eval_duration': 792856000, 'eval_count': 126, 'eval_duration': 5879021000, 'logprobs': None, 'model_name': 'llama3.1', 'model_provider': 'ollama'}, id='lc_run--019f1bf5-9935-7550-a0b8-d210e31e3a7b-0', tool_calls=[], invalid_tool_calls=[], usage_metadata={'input_tokens': 216, 'output_tokens': 126, 'total_tokens': 342}})]}
```

[AGENT FINAL RESPONSE]:

Based on the output of the vision tool, it appears that the image at the specified file path contains a picture of a golden retriever. The top prediction is 'golden retriever' with 56.02% confidence, indicating that this is the most likely object in the image. The top-3 predictions also include a n otterhound and a clumber, but these are less likely objects in the image.

Therefore, the answer to your original question is:

The asset located at file path: /Users/christisanderson/omnimind-project/data/sample\_image.jpg contains a picture of a golden retriever.

(.venv) %n@%m %1~ %# \$█

Now do MobileNetV2 fine-tuning instead of only generic ImageNet inference when there are labeled images found in class folders.

First, do a quick POC by running a bootstrap vision dataset loader to load a tiny smoke-test dataset structure:

The screenshot displays a VS Code editor interface for a project named 'omnimind-project'. The Explorer panel on the left shows the project's file structure, with the 'vision' directory under 'data' selected. The 'vision' directory contains 'train' and 'val' subdirectories, each with 'dog' and 'dota' folders containing image files. The main editor area shows a README.md file with instructions for generating serialized models and fine-tuning MobileNetV2. The terminal at the bottom shows the execution of the command 'python train\_artifacts.py --bootstrap-vision-dataset' and the subsequent fine-tuning of MobileNetV2, displaying progress bars and accuracy/loss metrics for each epoch.

```
64 # Imaging
69 2. Generate the serialized models required for the tools layer: bash:
70 python train_artifacts.py
71 - To create a tiny smoke-test dataset first, run:
72 - python train_artifacts.py --bootstrap-vision-dataset
73 - For task-specific vision accuracy, place labeled images in:
74 - data/vision/train/<class_name>/
75 - data/vision/val/<class_name>/
76 - Then rerun train_artifacts.py to fine-tune MobileNetV2 and save:
77 - data/models/vision_model.keras
78 - data/models/vision_labels.json
79 3. Prepare the local models:
```

```
(.venv) %n@m %1~ %# python train_artifacts.py --bootstrap-vision-dataset
[INIT] Bootstrapping tiny sample vision dataset...
[SUCCESS] Tiny sample dataset created under /Users/christisanderson/omnimind-project/data/vision
[NOTE] This dataset is for smoke tests only; replace it with real labeled images for useful accuracy.
[INIT] Training and serializing Scikit-Learn Model...
[SUCCESS] Scikit-Learn saved to /Users/christisanderson/omnimind-project/data/models/housing_model.joblib
[INIT] Preparing TensorFlow vision pipeline...
Found 2 files belonging to 2 classes.
Found 2 files belonging to 2 classes.
[INIT] Found 2 classes: dog, dota
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/mobilenet_v2_weights_tf_dim_ordering_tf_kernels_1.0_224_no_top.h5
9406464/9406464 0s 0us/step
[INIT] Training classification head...
/Users/christisanderson/omnimind-project/.venv/lib/python3.12/site-packages/keras/src/trainers/epoch_iterator.py:74: UserWarning: `shuffle=True` was passed, but will be ignored since the data `x` was provided as a tf.data.Dataset. The Dataset is expected to already be shuffled (via `.shuffle(buffer_size)`).
 self.data_adapter = data_adapters.get_data_adapter(
Epoch 1/5
1/1 2s 2s/step - accuracy: 0.5000 - loss: 0.8824 - val_accuracy: 0.5000 - val_loss: 0.8230
Epoch 2/5
1/1 0s 47ms/step - accuracy: 0.5000 - loss: 0.9013 - val_accuracy: 0.5000 - val_loss: 0.6927
Epoch 3/5
1/1 0s 47ms/step - accuracy: 0.0000e+00 - loss: 1.3260 - val_accuracy: 0.5000 - val_loss: 0.6312
Epoch 4/5
1/1 0s 48ms/step - accuracy: 0.5000 - loss: 0.7272 - val_accuracy: 1.0000 - val_loss: 0.6037
Epoch 5/5
1/1 0s 48ms/step - accuracy: 1.0000 - loss: 0.4521 - val_accuracy: 0.5000 - val_loss: 0.6223
[INIT] Fine-tuning upper MobileNetV2 layers...
Epoch 1/5
1/1 4s 4s/step - accuracy: 0.5000 - loss: 0.7515 - val_accuracy: 0.5000 - val_loss: 0.6234
Epoch 2/5
1/1 0s 55ms/step - accuracy: 0.5000 - loss: 0.6883 - val_accuracy: 0.5000 - val_loss: 0.6225
Epoch 3/5
1/1 0s 55ms/step - accuracy: 0.5000 - loss: 0.5793 - val_accuracy: 0.5000 - val_loss: 0.6218
Epoch 4/5
1/1 0s 55ms/step - accuracy: 0.5000 - loss: 0.4856 - val_accuracy: 0.5000 - val_loss: 0.6207
Epoch 5/5
1/1 0s 60ms/step - accuracy: 1.0000 - loss: 0.3123 - val_accuracy: 0.5000 - val_loss: 0.6195
[SUCCESS] TensorFlow model saved to /Users/christisanderson/omnimind-project/data/models/vision_model.keras
[SUCCESS] Vision labels saved to /Users/christisanderson/omnimind-project/data/models/vision_labels.json

[COMPLETE] All mock enterprise pipeline dependencies are generated.
(.venv) %n@m %1~ %#
```

**Added error to inform for the need to add required minimum number of unique images for each class:** ValueError: Vision dataset validation failed. Fine-tuning was skipped because the dataset does not meet minimum distinct-image requirements

```
(.venv) %n@m %1~ %# python train_artifacts.py
[INIT] Training and serializing Scikit-Learn Model...
[SUCCESS] Scikit-Learn saved to /Users/christisanderson/omnimind-project/data/models/housing_model.joblib
[INIT] Preparing TensorFlow vision pipeline...
Traceback (most recent call last):
 File "/Users/christisanderson/omnimind-project/train_artifacts.py", line 249, in <module>
 generate_tensorflow_artifact()
    ~~~~~
  File "/Users/christisanderson/omnimind-project/train_artifacts.py", line 162, in generate_tensorflow_artifact
    _validate_vision_dataset()
    ~~~~~
 File "/Users/christisanderson/omnimind-project/train_artifacts.py", line 152, in _validate_vision_dataset
 raise ValueError(
ValueError: Vision dataset validation failed. Fine-tuning was skipped because the dataset does not meet minimum distinct-image requirements: class 'dog' has only 1 distinct training images; minimum is 5; class 'dog' has only 1 distinct validation images; minimum is 2; class 'dog' reuses 1 identical image(s) across train and val; class 'dots' has only 1 distinct training images; minimum is 5; class 'dots' has only 1 distinct validation images; minimum is 2; class 'dots' reuses 1 identical image(s) across train and val.
(.venv) %n@m %1~ %#
```

**Fixed**

The screenshot displays the Visual Studio Code interface for the 'omnimind-project'. The Explorer sidebar on the left shows the project structure, including folders like \_\_pycache\_\_, .venv, data, models, vision, train, dog, and dots. The main editor area shows a README.md file with a diagram of the System Architecture. The diagram shows a User / Client API interacting with a System Architecture. The terminal at the bottom shows the output of the python train\_artifacts.py command, which includes training progress, accuracy, and loss metrics. The terminal output shows that the training was successful and the model was saved.

```
(.venv) %n@m %1~ %#
(.venv) %n@m %1~ %# python train_artifacts.py
[INIT] Training and serializing Scikit-Learn Model...
[SUCCESS] Scikit-Learn saved to /Users/christisanderson/omnimind-project/data/models/housing_model.joblib
[INIT] Preparing TensorFlow vision pipeline...
Found 10 files belonging to 2 classes.
Found 4 files belonging to 2 classes.
[INIT] Found 2 classes: dog, dots
[INIT] Training classification head...
/Users/christisanderson/omnimind-project/.venv/lib/python3.12/site-packages/keras/src/trainers/epoch_iterator.py:74: UserWarning: `shuffle=True` was passed, but will be ignored since the data `x` was provided as a tf.data.Dataset. The Dataset is expected to already be shuffled (via `.shuffle(buffer_size)`).
 self.data_adapter = data_adapters.get_data_adapter(
Epoch 1/5
1/1 ██████████ 2s 2s/step - accuracy: 0.4000 - loss: 1.1025 - val_accuracy: 0.0000e+00 - val_loss: 1.3801
Epoch 2/5
1/1 ██████████ 0s 111ms/step - accuracy: 0.4000 - loss: 0.9817 - val_accuracy: 0.2500 - val_loss: 1.2372
Epoch 3/5
1/1 ██████████ 0s 110ms/step - accuracy: 0.8000 - loss: 0.5584 - val_accuracy: 0.2500 - val_loss: 1.1117
Epoch 4/5
1/1 ██████████ 0s 111ms/step - accuracy: 0.8000 - loss: 0.3713 - val_accuracy: 0.5000 - val_loss: 1.0067
Epoch 5/5
1/1 ██████████ 0s 109ms/step - accuracy: 0.8000 - loss: 0.5608 - val_accuracy: 0.5000 - val_loss: 0.9144
[INIT] Fine-tuning upper MobileNetV2 layers...
Epoch 1/5
1/1 ██████████ 4s 4s/step - accuracy: 0.9000 - loss: 0.6327 - val_accuracy: 0.5000 - val_loss: 0.9016
Epoch 2/5
1/1 ██████████ 0s 135ms/step - accuracy: 0.6000 - loss: 0.5867 - val_accuracy: 0.5000 - val_loss: 0.8894
Epoch 3/5
1/1 ██████████ 0s 131ms/step - accuracy: 0.7000 - loss: 0.5766 - val_accuracy: 0.5000 - val_loss: 0.8773
Epoch 4/5
1/1 ██████████ 0s 133ms/step - accuracy: 0.8000 - loss: 0.5046 - val_accuracy: 0.5000 - val_loss: 0.8658
Epoch 5/5
1/1 ██████████ 0s 138ms/step - accuracy: 0.7000 - loss: 0.5353 - val_accuracy: 0.5000 - val_loss: 0.8543
[EVAL] Per-class validation accuracy:
- dog: 0.00% (n=2)
- dots: 100.00% (n=2)
[SUCCESS] TensorFlow model saved to /Users/christisanderson/omnimind-project/data/models/vision_model.keras
[SUCCESS] Vision labels saved to /Users/christisanderson/omnimind-project/data/models/vision_labels.json

[COMPLETE] All mock enterprise pipeline dependencies are generated.
(.venv) %n@m %1~ %#
```



**Model was able to successfully categorize the image after applying fine-tuning:**

The image shows a VS Code editor window with the 'omnimind-project' open. The Explorer sidebar on the left shows a file tree with folders like 'data', 'models', 'train', and 'val', each containing various image files (e.g., 'dog\_001.jpg', 'dots\_001.jpg'). The main editor area is split into two panes. The top pane shows a terminal window with a large JSON log output from an AI model (llama3.1) analyzing an image file 'sample\_image.jpeg'. The log includes details about the image's content, confidence levels for predictions (Dog: 58.34%, Dots: 41.66%), and tool calls for vision inference. A small image of a dog is visible in the background of the terminal window. The bottom pane shows the 'sample\_image.jpeg' file, which is a photograph of a dog's face. The terminal output is a detailed JSON log, including a 'values' section with a 'messages' array containing a 'HumanMessage' and an 'AIMessage', and an 'updates' section with a 'messages' array containing an 'AIMessage'.